



# Sumário

---

1. Introdução e Objetivo
2. Estimativa de Tempo e Dedicção
3. Fundamentos Absolutos (Fase 0) — 2 meses
4. Python — Do Zero ao Intermediário (Fase 1) — 3 meses
5. Back-End com Python (Fase 2) — 4 meses
6. Banco de Dados Completo (Fase 3) — 2 meses
7. DevOps, Cloud e Infraestrutura (Fase 4) — 2 meses
8. Inteligência Artificial e LLMs (Fase 5) — 4 meses
9. Node.js como Segunda Stack (Fase 6 — Opcional) — 3 meses
10. Segurança de APIs — Em profundidade
11. Testes de Software
12. Message Brokers e Eventos
13. Observabilidade e Monitoramento
14. System Design
15. 5 Projetos Práticos do Portfólio
16. Livros Essenciais por Fase
17. Comunidades, Canais e Recursos Gratuitos
18. Carreira e Entrevistas Técnicas
19. Rotina de Estudos e Marcos de Progresso

# 1. Introdução e Objetivo

---

Este documento é a fusão de dois planos de estudo: o Plano Multistack original (Node.js, Python, PHP, Java — ~2.800 horas) e o plano focado revisado para o mercado atual. O resultado é um roteiro único, sequencial e detalhado, com foco principal em Python back-end + Inteligência Artificial, e Node.js como segunda stack opcional ao final.

A estratégia central é simples: dominar uma stack com profundidade antes de expandir para outras. O mercado valoriza muito mais um desenvolvedor que entrega APIs robustas em produção com IA integrada do que alguém com conhecimento superficial em quatro linguagens. Após alcançar o nível pleno em Python, aprender Node.js leva apenas 2 a 3 meses porque 80% do conhecimento já é transferível.

## Objetivos ao final deste plano:

- Desenvolver e publicar APIs REST robustas em Python com FastAPI e Django
- Dominar banco de dados relacional (PostgreSQL) e NoSQL (Redis, MongoDB)
- Containerizar e fazer deploy de aplicações com Docker, CI/CD e AWS
- Construir aplicações que integram LLMs (ChatGPT, Claude, modelos locais)
- Implementar sistemas de RAG (Retrieval-Augmented Generation) em produção
- Criar agentes de IA autônomos com ferramentas e memória
- Ter 5 projetos reais publicados no GitHub e 1 rodando em produção
- Estar apto a entrevistas técnicas de nível junior forte a pleno
- Ter Node.js como segunda stack funcional (fase opcional)

## 2. Estimativa de Tempo e Dedicação

O plano original multistack estimava 2.800 horas totais para dominar quatro stacks. Este plano consolidado, focado em Python + IA + Node.js opcional, reduz o caminho para o primeiro emprego sem abrir mão da profundidade.

| Fase   | Conteúdo                             | Horas Est.   | Duração (1h30/dia) |
|--------|--------------------------------------|--------------|--------------------|
| Fase 0 | Fundamentos absolutos                | ~90h         | 2 meses            |
| Fase 1 | Python do zero ao intermediário      | ~140h        | 3 meses            |
| Fase 2 | Back-end com Python (FastAPI/Django) | ~180h        | 4 meses            |
| Fase 3 | Banco de dados completo              | ~90h         | 2 meses            |
| Fase 4 | DevOps, Cloud, Infraestrutura        | ~90h         | 2 meses            |
| Fase 5 | IA, LLMs, RAG, Agentes               | ~180h        | 4 meses            |
|        | <b>TOTAL (Stack Principal)</b>       | <b>~770h</b> | <b>17 meses</b>    |
| Fase 6 | Node.js (opcional, segunda stack)    | ~180h        | +4 meses           |
|        | <b>TOTAL COM NODE.JS</b>             | <b>~950h</b> | <b>21 meses</b>    |

### Cenários de dedicação:

**1h30 por dia (mínimo sustentável):** Stack principal em ~17 meses, com Node.js em ~21 meses

**2h por dia (recomendado):** Stack principal em ~13 meses, com Node.js em ~16 meses

**3h por dia (acelerado):** Stack principal em ~9 meses, com Node.js em ~11 meses

**Fins de semana (4h sáb + 4h dom):** Pode substituir os dias da semana, mas consistência cai muito

➤ *IMPORTANTE: Com as Fases 0, 1 e 2 concluídas (~9 meses com 1h30/dia), você já pode candidatar-se a vagas de desenvolvedor júnior back-end Python. Não espere terminar o plano inteiro para entrar no mercado.*

## FASE 0

# Fundamentos Absolutos

Logica • Git • Linux • Ingles tecnico — ~2 meses / ~90 horas

## Objetivo da fase

Antes de tocar em qualquer framework ou linguagem específica, você precisa dominar os alicerces que são usados em absolutamente todas as stacks: lógica de programação, controle de versão com Git, terminal Linux e inglês técnico para leitura de documentação. Sem isso, o aprendizado nas fases seguintes será muito mais lento e frustrante.

## TÓPICOS DETALHADOS

### Lógica de Programação

- Condicionais: if, else, elif, switch
- Lacos de repeticao: for, while, do-while (conceito)
- Recursao basica: fatorial, fibonacci, torres de hanoi
- Funcoes: parametros, retorno, escopo de variaveis
- Vetores e matrizes: percorrer, buscar, ordenar
- Pseudocodigo e fluxogramas: representar algoritmos antes de codar

### Estruturas de Dados — Conceitos

- Listas lineares: arrays, listas ligadas
- Pilhas (Stack): LIFO, uso em chamadas de funcao
- Filas (Queue): FIFO, uso em processamento de tarefas
- Dicionarios / Hash maps: chave-valor, colisoes
- Arvores binarias: noção basica, percurso
- Grafos: conceito, BFS e DFS de forma intuitiva

### Git e GitHub

- Conceito de controle de versao e por que usar
- git init, git add, git commit, git status, git log
- Branches: git branch, git checkout, git merge
- Repositorios remotos: git remote, git push, git pull, git clone
- Pull Requests: fluxo de revisao de codigo no GitHub
- .gitignore: o que ignorar em cada tipo de projeto
- Conflitos de merge: como resolver na pratica

### Terminal Linux / Bash

- Navegacao: ls, cd, pwd, mkdir, rm, cp, mv
- Manipulacao de arquivos: cat, less, head, tail, grep, find
- Permissoes: chmod, chown, sudo
- Processos: ps, kill, top, htop
- Pipes e redirecionamento: |, >, >>, 2>
- Variaveis de ambiente: export, .bashrc, .zshrc, .env

- Scripts bash basicos: shebang, if, for, funcoes
- SSH: conexao com servidores remotos, chaves RSA

## Inglês Técnico para Devs

- Leia TODA documentacao em ingles desde o primeiro dia
- Documentacao oficial sempre supera tutoriais em portugues
- Use dicionario tecnico, nao tradutor automatico para conceitos
- Stack Overflow: como ler, interpretar e fazer boas perguntas
- GitHub: entender READMEs, issues, pull requests em ingles

## Noções de HTTP e Redes

- Como funciona a internet: cliente-servidor, requisicao-resposta
- Metodos HTTP: GET, POST, PUT, PATCH, DELETE
- Status codes: 2xx (sucesso), 3xx (redirect), 4xx (cliente), 5xx (servidor)
- Headers HTTP: Content-Type, Authorization, Accept, Cache-Control
- URL, URI, endpoint — diferencas
- Autenticacao basica: cookies, tokens JWT (conceito)
- HTTPS: o que é SSL/TLS e por que usar

### CURSOS — ORDEM DE ESTUDO

#### ● Logica de Programacao Essencial

DIO — [dio.me](https://dio.me) (GRATUITO)

~4h · Logica, fluxogramas, variaveis, condicionais, loops na pratica

#### ● Curso de Algoritmos — Gustavo Guanabara

Curso em Video — [cursoemvideo.com](https://cursoemvideo.com) (GRATUITO)

~16h · 3 modulos completos de logica e algoritmos com exercicios

#### ● Git e GitHub para Iniciantes

Curso em Video — [cursoemvideo.com](https://cursoemvideo.com) (GRATUITO)

~6h · Versionamento completo, branches, pull requests, conflitos

#### ● Linux para Desenvolvedores

Udemy — [Matheus Battisti](#) (PAGO ~R\$27)

~7h · Terminal, permissoes, scripts bash, SSH, navegacao avancada

#### ● Estrutura de Dados — Conceitos Essenciais

YouTube — [CanalCodigo Fonte TV](#) (GRATUITO)

Assista a playlist completa sobre estruturas de dados

➤ **REGRA DE OURO:** Abra um repositório no GitHub HOJE e commite tudo que escrever a partir de agora, mesmo exercícios simples de lógica. O histórico de commits conta muito para recrutadores.

## FASE 1

# Python — Do Zero ao Intermediário

Sintaxe • POO • Estruturas • Async • Testes — ~3 meses / ~140 horas

## Por que Python como primeira linguagem?

Python é a escolha mais estratégica para quem quer back-end com IA. Você usa a mesma linguagem para ambos os objetivos. O mercado de Python back-end é enorme e crescente. Todos os melhores frameworks de IA (LangChain, Hugging Face, PyTorch, LlamaIndex) são nativos em Python. Após dominar Python, aprender outra linguagem leva meses, não anos.

## MÓDULO 1.1 — PYTHON BÁSICO (MÊS 1)

### Sintaxe e tipos de dados

- Variáveis, constantes e tipagem dinâmica
- Tipos: int, float, str, bool, None, bytes
- Operadores: aritméticos, comparação, lógicos, bitwise
- f-strings, formatação de strings, métodos de string
- Type hints: como anotar tipos em Python moderno

### Estruturas de controle e funções

- if/elif/else, operador ternário
- for/while, break, continue, else em loops
- Funções: def, parâmetros, \*args, \*\*kwargs, retorno múltiplo
- Funções lambda e funções de alta ordem
- Escopo: local, enclosing, global, built-in (LEGB)

### Estruturas de dados nativas

- Listas: criação, slicing, métodos (append, pop, sort, reverse)
- Tuplas: imutabilidade, unpacking, quando usar
- Dicionários: criação, acesso, .get(), .items(), .keys(), .values()
- Sets: operações de conjunto, union, intersection, difference
- List comprehensions, dict comprehensions, set comprehensions
- Generator expressions: lazy evaluation, memória eficiente

### Arquivos e exceptions

- Leitura e escrita de arquivos: open(), with, modos
- try/except/else/finally: tratamento de erros correto
- Exceções built-in: TypeError, ValueError, KeyError, etc.
- Criar exceções customizadas herdando de Exception
- Módulos: import, from...import, \_\_init\_\_.py, pacotes
- Módulos essenciais da stdlib: os, sys, pathlib, json, re, datetime

## MÓDULO 1.2 — PYTHON INTERMEDIÁRIO (MÊS 2-3)

## Programação Orientada a Objetos (POO)

- Classes: `__init__`, atributos de instancia vs classe, `self`
- Heranca simples e multipla: `super()`, MRO (Method Resolution Order)
- Encapsulamento: atributos privados com `_`, `__`, `@property`
- Polimorfismo: duck typing, method overriding
- Metodos especiais (dunder): `__str__`, `__repr__`, `__len__`, `__eq__`, `__lt__`
- Dataclasses: `@dataclass`, `fields`, `post_init`
- Abstract classes com ABC: definir interfaces em Python

## Decoradores e funcoes avancadas

- Closures: funcoes que capturam o escopo externo
- Decoradores simples: funcao que recebe e retorna funcao
- Decoradores com parametros e `@wraps`
- Decoradores de classe e metodos: `@classmethod`, `@staticmethod`
- Iteradores e iteraveis: `__iter__`, `__next__`, `StopIteration`
- Geradores: `yield`, `yield from`, pipelines de dados

## Programacao assincrona

- Conceito: event loop, corrotinas, tarefas
- `async def`, `await`: como definir e chamar corrotinas
- `asyncio.gather()`, `asyncio.create_task()`, `asyncio.sleep()`
- Quando usar `async`: I/O bound vs CPU bound
- `httpx` e `aiohttp`: requisicoes HTTP assincronas
- `asyncio.Queue`: comunicacao entre corrotinas

## Tipagem avancada e ferramentas

- `mypy`: verificacao estatica de tipos
- `Pydantic v2`: validacao de dados, modelos, coercao de tipos
- `Poetry` e `pyproject.toml`: gerenciamento moderno de dependencias
- Virtual environments: `venv`, como isolar projetos
- Linters: `ruff`, `flake8`; formatadores: `black`, `isort`
- Profiling basico: `cProfile`, `time`, `memory_profiler`

## CURSOS — ORDEM DE ESTUDO

### ● Python 3 do Mundo Real — Modulos 1, 2 e 3

[Curso em Video — Guanabara \(GRATUITO\)](#)

~40h divididos em 3 modulos. Base solida e didatica excepcional em PT-BR

### ● Python Impressionador: do Zero ao Avancado

[Udemy — Hashtag Treinamentos \(PAGO ~R\\$27\)](#)

~70h · POO completo, arquivos, APIs, automacoes, projetos praticos reais

### ● Estrutura de Dados e Algoritmos em Python

[Udemy — Rodrigo Fernandes \(PAGO ~R\\$27\)](#)

~10h · Listas ligadas, pilhas, filas, arvores, Big O Notation

### ● Python Fluente — Livro (Luciano Ramalho)

[Novatec / O'Reilly \(PAGO — referencia permanente\)](#)

Leia em paralelo a partir do capitulo 1. O melhor livro Python em portugues.



● **Programacao Assincrona com asyncio**

**Documentacao oficial [docs.python.org](https://docs.python.org) + [ArjanCodes YouTube](#)**

Assista a serie 'AsyncIO in Python' do ArjanCodes (YouTube, GRATUITO)

➤ **PRÁTICA OBRIGATÓRIA:** *Resolva no mínimo 50 exercícios no LeetCode (nível Easy) e 30 no Beecrowd durante esta fase. 15 minutos por dia de algoritmos é suficiente — o importante é a constância.*

## FASE 2

# Back-End com Python

FastAPI • Django DRF • APIs REST • Arquitetura • Autenticacao — ~4 meses / ~180h

## MÓDULO 2.1 — HTTP E APIS REST EM PROFUNDIDADE

- Principios REST: recursos, verbos, stateless, representacoes
- Design de endpoints: nomenclatura, versionamento (/v1/), hierarquia
- Idempotencia: quais metodos sao idempotentes e por que importa
- Paginacao: cursor-based vs offset-based, quando usar cada um
- Filtragem, ordenacao e busca em APIs: query parameters boas praticas
- Rate limiting: conceito, headers (X-RateLimit-\*), implementacao
- HATEOAS: hipermedia como o motor do estado da aplicacao
- OpenAPI e Swagger: documentar APIs de forma automatica e padronizada
- Versionamento de APIs: URI vs header vs query param
- Webhook: o que e, como implementar, como garantir entrega

## MÓDULO 2.2 — FASTAPI (FRAMEWORK PRINCIPAL)

### Setup e fundamentos

- Instalacao, estrutura de projeto recomendada
- Path parameters, query parameters, request body com Pydantic
- Response models: como definir o shape da resposta
- Status codes e HTTPException
- Dependency Injection: Depends(), como funciona por baixo
- Background tasks: executar codigo pos-resposta

### Autenticacao e autorizacao

- JWT: geracao com python-jose ou PyJWT, validacao, refresh token
- OAuth2PasswordBearer: fluxo de login padrao
- Permissoes e roles: como implementar RBAC simples
- API keys: autenticar servicos externos
- Lifespan e startup events: inicializacao de conexoes

### Banco de dados com FastAPI

- SQLAlchemy 2.x com async support (AsyncSession)
- Alembic: migrations versionadas, como criar e rodar
- Relacionamentos: 1:1, 1:N, N:N com tabela associativa
- Repositorio pattern: separar logica de acesso a dados
- Testcontainers: banco real em testes de integracao

### Funcionalidades avancadas

- Middlewares: logging de requisicoes, CORS, timing
- WebSockets com FastAPI: conexao persistente, broadcast
- Server-Sent Events (SSE): streaming de dados ao cliente

- Upload de arquivos: multipart/form-data, validacao, S3
- Cache com Redis: respostas cacheadas, invalidacao
- Configuracao com Pydantic Settings: .env, environments

## MÓDULO 2.3 — DJANGO + DJANGO REST FRAMEWORK

### Django core

- MTV: Models, Templates, Views (e como difere do MVC)
- ORM do Django: models, QuerySets, managers customizados
- Migrations: makemigrations, migrate, squashmigrations
- Admin do Django: customizacao, list\_display, filtros, acoes
- Django signals: pre\_save, post\_save, m2m\_changed
- Settings modular: base.py, development.py, production.py

### Django REST Framework (DRF)

- Serializers: ModelSerializer, validacao, nested serializers
- ViewSets e Routers: CRUD automatico com minimo de codigo
- Permissions: IsAuthenticated, IsAdminUser, permissao customizada
- Filtros: django-filter, SearchFilter, OrderingFilter
- Paginacao: PageNumberPagination, CursorPagination
- Throttling: limitar requisicoes por usuario/IP
- JWT com djangorestframework-simplejwt: login, refresh, blacklist

### Arquitetura e boas praticas

- Clean Architecture em Python: separar dominio, aplicacao, infra
- Principios SOLID aplicados a Python: exemplos praticos
- Design Patterns: Repository, Service Layer, Factory, Observer
- Separacao de concerns: views finas, logica em services
- Tratamento de erros centralizado: exception handlers globais

## CURSOS — ORDEM DE ESTUDO

#### ● FastAPI do Zero — Dunossauro

[fastapidozero.dunossauro.com](https://fastapidozero.dunossauro.com) (GRATUITO)

Melhor curso gratuito de FastAPI em PT-BR. Cobre do basico ao deploy com CI/CD

#### ● Django REST Framework: APIs com Python

[Udemy — Geek University](#) (PAGO ~R\$27)

~12h · DRF completo: serializers, viewsets, autenticacao, permissoes, filtros

#### ● Arquitetura Limpa e SOLID em Python

[Udemy — Otavio Miranda](#) (PAGO ~R\$27)

~10h · Principios de design, padroes GoF, Clean Architecture na pratica

#### ● Microservicos com Python e FastAPI

[Udemy](#) (PAGO ~R\$27)

~8h · RabbitMQ, comunicacao entre servicos, separacao de responsabilidades

● **Clean Architecture — Robert C. Martin (Livro)**

[Alta Books](#) / [O'Reilly \(PAGO — referencia\)](#)

Leia em paralelo a esta fase. Fundamental para nível pleno.

➤ *PROJETO DESTA FASE: Construa uma API completa com autenticação JWT, CRUD de recursos reais, relacionamentos no banco, documentação Swagger automática, testes com pytest e Docker Compose. Publique no GitHub com README detalhado. Este é o projeto mais importante do seu portfólio inicial.*

## FASE 3

# Banco de Dados Completo

SQL • PostgreSQL • SQLAlchemy • Redis • MongoDB — ~2 meses / ~90h

## MÓDULO 3.1 — SQL E POSTGRESQL

### SQL Completo

- DDL: CREATE, ALTER, DROP — criação e modificação de esquemas
- DML: SELECT, INSERT, UPDATE, DELETE — manipulação de dados
- Joins: INNER, LEFT, RIGHT, FULL OUTER, CROSS, self-join
- Subqueries: correlacionadas, no FROM, no WHERE, no SELECT
- Funções de agregação: COUNT, SUM, AVG, MAX, MIN, GROUP BY, HAVING
- Window functions: ROW\_NUMBER, RANK, LEAD, LAG, PARTITION BY
- CTEs (Common Table Expressions): WITH, recursivas
- Transactions: BEGIN, COMMIT, ROLLBACK, SAVEPOINT
- ACID: Atomicidade, Consistência, Isolamento, Durabilidade
- Níveis de isolamento: READ UNCOMMITTED até SERIALIZABLE

### PostgreSQL Avançado

- Tipos especiais: JSONB, arrays, UUID, ENUM, range types
- Índices: B-tree, Hash, GIN, BRIN — quando usar cada um
- EXPLAIN ANALYZE: ler o plano de execução de queries
- Conexão pooling: PgBouncer, por que e necessário
- Particionamento de tabelas: range, list, hash
- Full-text search nativo: to\_tsvector, to\_tsquery
- Funções e procedures: PL/pgSQL básico
- Triggers: antes e depois de DML, casos de uso
- Replicação: conceito de primário/replica, read replicas

### Modelagem de Dados

- Normalização: 1FN, 2FN, 3FN — quando normalizar e quando não
- Desnormalização: quando sacrificar pureza por performance
- Relacionamentos: 1:1, 1:N, N:N com tabela de junção
- Chaves naturais vs surrogadas: UUID vs serial vs bigint
- Soft delete: deleted\_at vs tabela de histórico
- Auditoria: created\_at, updated\_at, created\_by, updated\_by

## MÓDULO 3.2 — ORM E MIGRATIONS

- SQLAlchemy 2.x: declarative\_base, mapped\_column, relationship()
- Sessions: Session vs AsyncSession, unit of work pattern
- Queries com SQLAlchemy: select(), where(), join(), scalars()
- Lazy vs eager loading: selectinload, joinedload, quando usar
- Alembic: autogenerate, downgrade, merge de branches de migration
- Django ORM: select\_related, prefetch\_related, annotate, aggregate

- N+1 problem: como identificar e resolver em qualquer ORM

### MÓDULO 3.3 — REDIS (CACHE E FILAS)

- Estruturas de dados do Redis: strings, lists, sets, sorted sets, hashes
- Expiração de chaves: TTL, EXPIRE, EXPIREAT
- Cache-aside pattern: read-through, write-through, write-behind
- Pub/Sub do Redis: publicar e subscrever a canais em tempo real
- Redis como fila: LPUSH/RPOP, BLPOP para consumo bloqueante
- Redis com Python: redis-py e aioredis (async)
- Celery + Redis: filas de tasks assíncronas, beat scheduler
- Redis Streams: alternativa mais robusta ao Pub/Sub para eventos
- Persistência: RDB snapshots vs AOF (Append Only File)

### MÓDULO 3.4 — MONGODB (NOSQL DOCUMENTAL)

- Modelo de dados: documentos BSON, collections, databases
- CRUD com PyMongo: insert\_one, find, update\_many, delete\_one
- Queries: filtros, projecao, sort, limit, skip
- Índices no MongoDB: single, compound, text, geoespacial
- Aggregation pipeline: \$match, \$group, \$project, \$lookup, \$unwind
- Motor (async): PyMongo assíncrono para usar com FastAPI
- MongoDB Atlas: cluster gratuito na nuvem, Data API
- Quando usar MongoDB vs PostgreSQL: casos de uso reais
- Schema validation com MongoDB: JSONSchema dentro do banco

### CURSOS — ORDEM DE ESTUDO

#### ● SQL completo e MySQL

[Curso em Vídeo — Guanabara \(GRATUITO\)](#)

~40h em módulos. Base sólida de SQL, joins avançados, subqueries, views

#### ● PostgreSQL do Zero ao Avançado

[Udemy \(PAGO ~R\\$27\)](#)

~10h · PostgreSQL, indexação, EXPLAIN ANALYZE, particoes, funcoes

#### ● SQLAlchemy e Alembic com Python

[FastAPI do Zero \(Fase 2\) + Documentacao oficial](#)

Ja coberto no curso de FastAPI. Complementar com documentacao oficial.

#### ● Redis — Banco de Dados em Memoria

[DIO ou Udemy \(PAGO ~R\\$27\)](#)

~5h · Todas as estruturas, cache, pub/sub, Celery na pratica

#### ● MongoDB com Python (PyMongo + Motor)

[MongoDB University — mongodb.com/learn \(GRATUITO\)](#)

~6h · Certificacao gratuita oficial da MongoDB. Altamente recomendado.

#### ● Designing Data-Intensive Applications (Livro)

[O'Reilly — Martin Kleppmann \(PAGO — referencia\)](#)

Melhor livro sobre bancos e sistemas distribuidos. Leia nesta fase.

➤ *PRÁTICA: Use o site [sqlzoo.net](https://sqlzoo.net) e [mode.com/sql](https://mode.com/sql) para praticar SQL interativamente. Para MongoDB, crie um cluster gratuito no Atlas e importe datasets de exemplo para praticar queries de agregacao.*

## FASE 4

# DevOps, Cloud e Infraestrutura

Docker • CI/CD • AWS • Kubernetes basico • Monitoramento — ~2 meses / ~90h

## MÓDULO 4.1 — DOCKER E CONTAINERS

### Conceitos fundamentais

- O que são containers vs máquinas virtuais: diferenças reais
- Imagens Docker: layers, copy-on-write, como são construídas
- Docker Hub e registros privados: push, pull, tags
- Volumes: named volumes, bind mounts, tmpfs — quando usar cada
- Redes Docker: bridge, host, overlay — comunicação entre containers

### Dockerfile — boas práticas

- Escolha da imagem base: python:3.12-slim vs python:3.12-alpine
- Multi-stage builds: imagem de build vs imagem final menor
- Ordem das instruções para melhor cache: COPY requirements antes do código
- COPY vs ADD: quando usar cada um
- Variáveis ENV e ARG: diferenças e casos de uso
- HEALTHCHECK: definir verificação de saúde do container
- Usuário não-root: segurança em produção

### Docker Compose

- services, volumes, networks: estrutura completa do compose
- depends\_on e healthcheck: garantir ordem de inicialização
- Profiles: diferentes configurações para dev e produção
- Secrets no Compose: como não hardcodar credenciais
- Compose com hot-reload para desenvolvimento local

## MÓDULO 4.2 — CI/CD COM GITHUB ACTIONS

- Conceito de pipeline: integrar e entregar de forma contínua
- GitHub Actions: workflows, jobs, steps, actions do marketplace
- Trigger de workflows: push, pull\_request, schedule, workflow\_dispatch
- Rodar testes automaticamente em cada push para main
- Build e push de imagem Docker para Docker Hub ou ECR
- Deploy automático em servidor VPS via SSH
- Secrets e environments no GitHub: NEVER commitar credenciais
- Environments com aprovação manual: staging e production gates
- Cache de dependências Python no GitHub Actions
- Matrix builds: testar em múltiplas versões do Python

## MÓDULO 4.3 — AWS (FOCO EM SERVIÇOS ESSENCIAIS)

### Compute e Deploy

- EC2: instâncias, tipos, security groups, key pairs, Elastic IP



- Configurar VPS Ubuntu: instalar Docker, Nginx, Python
- ECS (Elastic Container Service): deploy de containers gerenciado
- AWS Lambda: funcoes serverless, quando faz sentido usar

## Storage e Banco de Dados

- S3: buckets, permissões, ACLs, signed URLs, lifecycle rules
- RDS: PostgreSQL gerenciado, backups automaticos, multi-AZ
- ElastiCache: Redis gerenciado na AWS

## Rede e Seguranca

- VPC: subnets publicas e privadas, Internet Gateway, NAT Gateway
- IAM: usuarios, roles, policies — principio do menor privilegio
- Secrets Manager: armazenar credenciais de forma segura
- Route 53: DNS, registrar dominio, apontar para EC2
- ACM: certificados SSL/TLS gratuitos para HTTPS

## Nginx como proxy reverso

- Configurar Nginx para redirecionar porta 80/443 para a API
- HTTPS com Certbot (Let's Encrypt): certificado gratuito
- Rate limiting no Nginx: limitar requisicoes por IP
- Gzip compression: compressao de respostas
- Logs de acesso e erro do Nginx: onde ficam, como ler

## MÓDULO 4.4 — KUBERNETES (BÁSICO PARA PLENO)

- O que é Kubernetes e por que existe: orquestracao de containers
- Pods: a menor unidade deployavel no K8s
- Deployments: declarar estado desejado, rolling updates, rollback
- Services: ClusterIP, NodePort, LoadBalancer — exposicao de pods
- Ingress: roteamento HTTP/S para services, TLS termination
- ConfigMaps: configuracao externalizada sem rebuild da imagem
- Secrets: dados sensiveis no K8s (base64, nao criptografado por padrao)
- kubectl essencial: get, describe, logs, exec, apply, delete
- Helm basico: o que são charts, como instalar dependencias
- Minikube: rodar K8s localmente para aprender

## CURSOS — ORDEM DE ESTUDO

### ● Docker para Desenvolvedores

[Udemy — Matheus Battisti \(PAGO ~R\\$27\)](#)

~12h · Containers, volumes, redes, Compose, registry, boas praticas

### ● GitHub Actions: CI/CD na Pratica

[DIO ou Udemy \(PAGO ~R\\$27\)](#)

~5h · Workflows, testes automaticos, deploy automatico via SSH

### ● AWS Cloud Practitioner Essentials

[AWS Skill Builder — skillbuilder.aws \(GRATUITO\)](#)

~6h · Fundamentos AWS, servicos principais, billing, seguranca basica

● **Deploy de APIs Python com Docker + AWS**

[Udemy \(PAGO ~R\\$27\)](#)

~8h · EC2, RDS, S3, Nginx, HTTPS, variaveis de ambiente na pratica

● **Kubernetes para Desenvolvedores**

[Udemy — Matheus Battisti ou KodeKloud \(PAGO ~R\\$27\)](#)

~8h · Pods, Deployments, Services, Ingress, kubectl, Helm basico

➤ *META DESTA FASE: Pegar o projeto da Fase 2 e colocá-lo no ar de verdade com domínio próprio, HTTPS, Docker em produção e CI/CD automático no GitHub Actions. Um projeto rodando em produção é ouro no portfólio.*

## Por que IA é parte do plano de back-end?

Em 2025 e além, desenvolvedores back-end que sabem integrar LLMs ganham salários até 40% maiores e têm muito mais oportunidades. O mercado não quer apenas quem usa ChatGPT via API — quer quem consegue construir sistemas robustos com RAG, memória, ferramentas e avaliação de qualidade. Esta fase te coloca nessa posição.

### MÓDULO 5.1 — FUNDAMENTOS DE ML COM PYTHON

#### NumPy e Pandas

- NumPy: arrays, operacoes vetorizadas, broadcasting, indexacao
- Pandas: Series, DataFrame, leitura de CSV/JSON, groupby, merge
- Matplotlib e Seaborn: visualizacao de dados basica
- Tratamento de dados: missing values, outliers, normalizacao

#### Machine Learning com Scikit-learn

- Conceitos: supervisionado, nao supervisionado, reforco
- Regressao linear e logistica: como e quando usar
- Arvores de decisao, Random Forest, Gradient Boosting
- Classificacao: KNN, SVM, Naive Bayes
- Clustering: K-Means, DBSCAN
- Pipeline do sklearn: preprocessamento + modelo em um objeto
- Cross-validation: como avaliar modelos corretamente
- Metricas: accuracy, precision, recall, F1, ROC-AUC, RMSE

#### Redes Neurais — conceitos fundamentais

- Perceptron: neuronio artificial, funcao de ativacao
- Feedforward network: forward pass, loss function
- Backpropagation: como gradientes fluem para tras
- Optimizadores: SGD, Adam, RMSprop — diferencas praticas
- Regularizacao: dropout, L1/L2, batch normalization
- CNN: convolucao, pooling, para imagens
- RNN e LSTM: para sequencias e texto (conceito historico)

### MÓDULO 5.2 — LARGE LANGUAGE MODELS (LLMS)

#### Como LLMs funcionam

- Transformer architecture: attention mechanism, encoder-decoder
- Tokenizacao: BPE, WordPiece, como texto vira numeros
- Embeddings: representacao vetorial de palavras e frases
- Context window: limite de tokens, como gerenciar
- Temperature, top-p, top-k: como controlar aleatoriedade

- Pre-training vs fine-tuning vs few-shot learning

## Prompt Engineering avançado

- Zero-shot, one-shot, few-shot: quando usar cada um
- Chain-of-thought (CoT): induzir raciocínio passo a passo
- ReAct: reasoning + acting em agentes
- Structured outputs: forçar saída em JSON com schemas
- Function calling / Tool use: LLM chamando funções externas
- System prompts: como escrever instruções eficazes
- Prompt injection: ataques e defesas

## APIs de LLMs na prática

- OpenAI API: chat completions, embeddings, function calling
- Anthropic Claude API: Messages API, system prompts, tool use
- Google Gemini API: multimodal, context longo
- Modelos locais com Ollama: rodar LLMs sem custo na sua máquina
- Gerenciamento de custos: tokens, pricing, como otimizar
- Streaming de respostas: como fazer o LLM responder em tempo real

## MÓDULO 5.3 — RAG E BANCOS VETORIAIS

- Embeddings de texto: como gerar com OpenAI, Sentence Transformers
- Similaridade cosseno: busca por documentos semanticamente próximos
- Bancos vetoriais: Pinecone, Weaviate, Chroma, pgvector (PostgreSQL)
- Chunking de documentos: estratégias, tamanho, sobreposição
- Pipeline RAG completo: ingest -> embed -> store -> retrieve -> generate
- Metadata filtering: filtrar chunks por data, autor, categoria
- Reranking: ordenar resultados de busca com um modelo específico
- Hybrid search: combinar busca semântica com BM25 (lexica)
- RAGAS: framework para avaliar qualidade de sistemas RAG
- Multi-modal RAG: trabalhar com PDFs, imagens, tabelas

## MÓDULO 5.4 — LANGCHAIN, LLAMAINDEX E AGENTES

### LangChain

- Chains: sequência de operações com LLMs
- Prompts: PromptTemplate, ChatPromptTemplate, few-shot templates
- Memory: ConversationBufferMemory, ConversationSummaryMemory
- Retrievers: como buscar documentos relevantes
- LCEL (LangChain Expression Language): composição de chains

### LangGraph — Agentes Avançados

- Graph-based workflows: nodes, edges, condições
- State management: como manter estado entre steps
- Tool calling: definir e usar ferramentas (busca web, banco, APIs)
- Human-in-the-loop: pausar agente para aprovação humana

- Multi-agent systems: varios agentes colaborando
- Checkpointing: persistir e retomar estado de agentes

## MLOps basico para LLMs

- MLflow: rastreamento de experimentos, versao de modelos
- Serving modelos com FastAPI: criar endpoint para inferencia
- Monitoramento de LLMs: latencia, custo, qualidade de respostas
- Model drift: como detectar degradacao de qualidade ao longo do tempo
- LangSmith: observabilidade especifica para aplicacoes LangChain

### CURSOS — ORDEM DE ESTUDO

#### Machine Learning e Data Science com Python

[Udemy — Jones Granatyr \(PAGO ~R\\$27\)](#)

~70h · NumPy, Pandas, sklearn completo, redes neurais, projetos praticos

#### Deep Learning Specialization (5 cursos)

[Coursera — Andrew Ng \(GRATIS para auditar\)](#)

~80h · Redes neurais, CNN, RNN, otimizacao, backprop — o melhor do mundo

#### Neural Networks: Zero to Hero — Karpathy

[YouTube — Andrej Karpathy \(GRATUITO\)](#)

Serie completa: construir GPT do zero em Python. Obrigatorio para entender LLMs

#### LangChain for LLM Application Development

[DeepLearning.AI — Andrew Ng \(GRATUITO\)](#)

~4h · Chains, agents, memory, RAG basico. deeplearning.ai

#### Building Systems with the ChatGPT API

[DeepLearning.AI \(GRATUITO\)](#)

~3h · Pipelines com LLMs, classificacao, moderacao, RAG

#### Vector Databases: from Embeddings to Apps

[DeepLearning.AI \(GRATUITO\)](#)

~3h · Weaviate, Pinecone, busca semantica, RAG completo na pratica

#### AI Agents in LangGraph

[DeepLearning.AI \(GRATUITO\)](#)

~4h · Agentes autonomos, tool use, planejamento, ReAct, LangGraph

#### Hugging Face NLP Course

[huggingface.co/learn \(GRATUITO\)](#)

Transformers, pipelines, modelos open source, fine-tuning basico

#### AI Engineering — Chip Huyen (Livro)

[O'Reilly 2025 \(PAGO — referencia obrigatoria\)](#)

Colocar IA em producao, RAG, avaliacao, MLOps. Melhor livro recente da area.

#### Build a LLM From Scratch — Raschka (Livro)

[Manning 2024 \(PAGO — referencia avancada\)](#)

Construir um GPT do zero em Python. Para entender LLMs por dentro.

➤ **PROJETO FINAL DA FASE 5: API back-end que integra um LLM com RAG completo (PDFs reais, banco vetorial, chunking, reranking), autenticação JWT, histórico de conversas no PostgreSQL, streaming de respostas e**

*deploy na AWS. Este projeto sozinho já garante entrevistas de nível pleno.*

## Node.js — Segunda Stack (Opcional)

JavaScript avançado • Express • NestJS • TypeScript — ~3-4 meses / ~180h

### Por que Node.js depois de Python?

Com Python dominado, aprender Node.js leva apenas 3 a 4 meses porque a lógica de APIs REST, banco de dados, Docker, CI/CD e arquitetura já é conhecida — você só troca a sintaxe. Node.js é a stack mais demandada no Brasil para vagas de back-end no geral, então ter as duas te torna extremamente competitivo. Esta fase é baseada diretamente no Plano Multistack original.

#### MÓDULO 6.1 — JAVASCRIPT AVANÇADO E NODE.JS NATIVO

##### JavaScript ES6+ (revisão focada)

- let/const vs var: escopo de bloco, hoisting, temporal dead zone
- Arrow functions: this lexico, quando usar e quando não usar
- Destructuring: arrays, objetos, default values, renaming
- Spread/rest operators: copiar objetos, juntar arrays, parametros
- Promises: .then(), .catch(), .finally(), Promise.all(), race()
- Async/await: como funciona por baixo, error handling com try/catch
- Modulos ES Modules: import/export vs CommonJS require()
- Optional chaining (?.) e nullish coalescing (??) operators

##### Node.js nativo — Event Loop e core

- Event Loop: call stack, microtask queue, macrotask queue
- Non-blocking I/O: como Node.js lida com concorrência
- CommonJS vs ES Modules: interoperabilidade, package.json type
- Filesystem: fs.readFile, fs.writeFile, fs.watch, streams
- Streams: Readable, Writable, Transform, pipeline()
- http/https nativo: criar servidor sem framework
- child\_process: spawn, exec, fork — rodar processos externos
- Worker Threads: paralelismo real para CPU-bound tasks
- Buffers e encoding: binary data, base64, UTF-8

##### TypeScript para Node.js

- Setup: tsconfig.json, strict mode, target, moduleResolution
- Tipos básicos: interfaces vs types, generics, utility types
- Decoradores: @Controller, @Injectable (NestJS style)
- Tipos para Express/NestJS: Request, Response, Next tipados
- ts-node e tsx: rodar TypeScript sem compilar em dev

#### MÓDULO 6.2 — EXPRESS.JS E NESTJS

##### Express.js

- Rotas: app.get(), app.post(), router.use(), params vs query
- Middlewares: ordem de execução, next(), error handling (4 args)

- Validacao: Joi ou Zod para validar request body
- Autenticacao JWT: jsonwebtoken, middleware de protecao de rotas
- Sequelize ou Prisma: ORM para PostgreSQL/MySQL
- Estrutura MVC: separar routes, controllers, services, models

### NestJS (framework recomendado para pleno)

- Modulos, Controllers, Providers: arquitetura modular
- Injecao de dependencia: como NestJS gerencia o container
- Guards: autorizacao declarativa com @UseGuards
- Interceptors: transformar respostas, logging, cache
- Pipes: validacao e transformacao com class-validator
- Prisma com NestJS: schema.prisma, migrations, PrismaService
- WebSockets com NestJS: @WebSocketGateway, eventos em tempo real
- Microservicos com NestJS: TCP, RabbitMQ, Kafka transporters

## MÓDULO 6.3 — DEPLOY E FERRAMENTAS NODE.JS

- PM2: process manager, cluster mode, monitoramento, logs rotativos
- Testes com Jest: unit tests, mocks, coverage, testes de integracao
- Supertest: testar endpoints HTTP sem subir servidor real
- Docker para Node.js: .dockerignore, node\_modules fora do COPY
- Logging com Pino ou Winston: logs estruturados em JSON
- Helmet: headers de seguranca para Express/NestJS
- express-rate-limit: limitar requisicoes por IP
- CORS configurado: origins, methods, credentials corretos
- Prisma Migrate: migrations versionadas para PostgreSQL
- Socket.io: WebSockets com fallback para long-polling

## CURSOS — ORDEM DE ESTUDO

### ● JavaScript: O Guia Definitivo — Livro

[Alta Books — David Flanagan \(PAGO — referencia\)](#)

A biblia do JavaScript. Leia os capitulos de ES6+ e assincronia primeiro.

### ● Node.js do Zero ao Avancado — Rocketseat

[Udemy ou Rocketseat \(PAGO ~R\\$27 a R\\$100\)](#)

~20h · Node nativo, Express, TypeScript, PostgreSQL, deploy basico

### ● NestJS do Zero ao Deploy

[Udemy \(PAGO ~R\\$27\)](#)

~15h · NestJS completo: modulos, DI, guards, Prisma, testes, Docker

### ● TypeScript do Zero ao Avancado

[Udemy — Otavio Miranda \(PAGO ~R\\$27\)](#)

~13h · Types, interfaces, generics, decoradores, configuracao

### ● Node.js Design Patterns — Livro

[Packt — Mario Casciaro \(PAGO — referencia avancada\)](#)

Padroes de projeto especificos para Node.js. Leia durante a fase.



# Tópicos Transversais — Aplicar em Todas as Fases

---

## Segurança de APIs

Segurança é cobrada em toda entrevista de nível pleno. Não é um tópico isolado — deve ser aplicado desde o primeiro projeto. Estude o OWASP Top 10 ao final da Fase 2 e aplique em todos os projetos seguintes.

### OWASP Top 10 — as 10 vulnerabilidades mais críticas

- Broken Access Control: verificar permissões em TODA requisição
- Cryptographic Failures: não armazenar senhas em texto puro (bcrypt)
- Injection: SQL injection, XSS — usar ORM e sanitizar entradas
- Insecure Design: threat modeling desde o início do projeto
- Security Misconfiguration: defaults inseguros, debug em produção
- Vulnerable Components: pip-audit, dependabot para dependências
- Authentication Failures: brute force, tokens fracos, logout correto
- Integrity Failures: verificar integridade de pipelines e pacotes
- Logging Failures: logar eventos de segurança, alertas
- SSRF: validar URLs fornecidas pelo usuário antes de requisitar

### Práticas de segurança no dia a dia

- HTTPS obrigatório: redirecionar HTTP para HTTPS sempre
- CORS configurado corretamente: não usar \* em produção
- JWT seguro: algoritmo RS256, expiração curta, revogação via blacklist
- Rate limiting: limitar tentativas de login, endpoints sensíveis
- Input validation: validar e sanitizar TUDO que vem do usuário
- Variáveis de ambiente: NUNCA comitar .env com credenciais
- Headers de segurança: HSTS, CSP, X-Frame-Options, X-Content-Type
- Secrets management: AWS Secrets Manager, HashiCorp Vault
- Least privilege: usuário do banco sem permissão de DROP TABLE
- Dependency scanning: atualizar dependências, checar CVEs

➤ *Recurso gratuito: leia o [OWASP Testing Guide completo em owasp.org](https://owasp.org) e o [Cheat Sheet Series](#). São as referências mais completas de segurança para desenvolvedores. Curso: [Segurança em APIs REST — Udemy](#) (~R\$27).*

## Testes de Software

### Pirâmide de testes — o que testar e como

- Testes unitários: testar funções e classes isoladas com mocks
- Testes de integração: testar interação entre componentes reais
- Testes end-to-end: simular usuário usando o sistema completo
- Testes de contrato: garantir que API mantém contrato com clientes
- Testes de carga: Locust — simular N usuários simultâneos na API

## pytest — ferramenta principal para Python

- Fixtures: setup e teardown de dados para testes
- Parametrize: rodar mesmo teste com diferentes entradas
- Mocking: unittest.mock, MagicMock, patch — simular dependencias
- pytest-asyncio: testar funcoes async/await
- httpx.AsyncClient: testar endpoints FastAPI sem subir servidor
- Testcontainers: banco PostgreSQL real em container nos testes
- coverage.py: medir cobertura de codigo, relatorios HTML
- TDD: Test-Driven Development — escrever teste antes do codigo

➤ *Livro: Python Testing with pytest — Brian Okken (O'Reilly). O melhor recurso de testes em Python. Leia durante a Fase 2.*

## Message Brokers e Arquitetura Orientada a Eventos

- Por que usar mensageria: desacoplamento, resiliencia, escalabilidade
- RabbitMQ: exchanges, queues, bindings, acknowledgments, dead letters
- Apache Kafka: topicos, partitions, consumer groups, offset management
- Quando usar RabbitMQ vs Kafka: task queues vs event streaming
- Celery com Redis/RabbitMQ: tasks assincronas em Python
- Event-driven architecture: publishers, subscribers, event schemas
- CQRS: separar comandos de consultas para escalabilidade
- Event Sourcing: armazenar eventos em vez de estado atual
- Outbox pattern: garantir que evento e banco sao atualizados juntos
- Saga pattern: transacoes distribuidas entre microservicos

➤ *Curso: Kafka for Beginners — Udemy, Stephane Maerek (~R\$27). O melhor instrutor do tema. Faça junto com microserviços na Fase 2.*

## Observabilidade e Monitoramento

### Os tres pilares da observabilidade

- Logs: registrar o que aconteceu — structlog, JSON logging, centralizacao
- Metrics: medir o que esta acontecendo — Prometheus, Grafana dashboards
- Traces: rastrear requisicoes entre servicos — OpenTelemetry, Jaeger

### Ferramentas praticas

- structlog: logs estruturados em Python com contexto automatico
- python-json-logger: formatar logs como JSON para ingestao
- Sentry: captura de excecoes em tempo real, stack traces, alertas
- Prometheus: coletar metricas da aplicacao e do servidor
- Grafana: dashboards de metricas, alertas, visualizacao
- ELK Stack: Elasticsearch + Logstash + Kibana para analise de logs
- OpenTelemetry: padrao aberto para instrumentacao de traces
- Uptime monitoring: UptimeRobot, Better Uptime (gratuito)

➤ *Curso: Monitoring and Alerting with Prometheus e Grafana — Udemy (~R\$27). Faça durante ou após a Fase 4.*

## System Design

Um dos tópicos mais cobrados em entrevistas de nível pleno e sênior. Como projetar sistemas que escalam para milhões de usuários. Não tem curso único — é um conjunto de conceitos que você aprende ao longo de todas as fases.

- Load balancing: round-robin, least connections, sticky sessions
- Horizontal vs vertical scaling: quando escalar cada um
- Caching strategies: CDN, application cache, database cache, browser cache
- CAP theorem: Consistencia, Disponibilidade, Tolerancia a Particao
- BASE vs ACID: sistemas distribuidos vs bancos relacionais
- Sharding de banco de dados: hash sharding, range sharding, problemas
- CDN: Content Delivery Network — servir arquivos estaticos globalmente
- Consistent hashing: como distribuir dados entre nos de forma eficiente
- Consistencia eventual: quando aceitar dados desatualizados
- API Gateway: ponto de entrada unico, autenticacao, rate limiting
- Circuit breaker: Resilience4j, como prevenir falhas em cascata
- Idempotent operations: garantir que requisicoes duplicadas nao causem problemas
- Database replication: read replicas, leader election, failover
- Message queues como buffer: absorver picos de trafego
- Microservices vs monolith: trade-offs, quando migrar, strangler fig

➤ *Recurso gratuito: System Design Primer — [github.com/donnmartin/system-design-primer](https://github.com/donnmartin/system-design-primer). Leia inteirinho antes de qualquer entrevista. Livro: Designing Data-Intensive Applications — Martin Kleppmann.*

## 5 Projetos Práticos do Portfólio

---

Esses 5 projetos formam um portfólio completo do júnior avançado ao pleno. Cada um deve ter README detalhado no GitHub explicando o que é, como rodar localmente e quais tecnologias foram usadas. Não precisa ser perfeito — precisa estar funcionando.

### Projeto 1 — API de gerenciamento de tarefas (To-do API)

Fase recomendada: Fase 1-2

CRUD completo de tarefas com usuários, autenticação JWT completa (login, refresh token, logout com blacklist), categorias, prioridades, filtros e ordenação. Documentação automática com Swagger. Paginação. Testes unitários e de integração com pytest. Docker Compose com PostgreSQL.

#### Stack tecnológica:

- FastAPI
- PostgreSQL
- SQLAlchemy
- Alembic
- JWT
- pytest
- Docker Compose

#### Diferenciais a implementar:

- Autenticação JWT com refresh token e blacklist no Redis
- Testes cobrindo pelo menos 80% do código
- Documentação Swagger completa e funcional
- Makefile com comandos úteis: make dev, make test, make migrate

---

### Projeto 2 — E-commerce API simplificado

Fase recomendada: Fase 2-3

Produtos, categorias, carrinho de compras, pedidos e pagamentos simulados (Stripe sandbox). Eventos assíncronos com Celery + Redis: envio de e-mail de confirmação, atualização de estoque. Busca full-text com PostgreSQL. Upload de imagens de produtos para S3. RBAC com roles admin/cliente.

#### Stack tecnológica:

- Django DRF
- PostgreSQL
- Redis
- Celery
- AWS S3
- Stripe API

#### Diferenciais a implementar:

- Tarefas assíncronas reais: email, notificações, relatórios
  - Upload de imagens com pre-signed URLs no S3
  - Busca full-text nativa do PostgreSQL
  - Testes de integração com banco real via Testcontainers
-

## Projeto 3 — Deploy completo em producao

Fase recomendada: Fase 4

Pegar o Projeto 1 ou 2 e colocar no ar de verdade: domínio próprio registrado, HTTPS com certificado SSL via Certbot, Nginx como proxy reverso, Docker em servidor EC2 da AWS, pipeline CI/CD com GitHub Actions que roda testes e faz deploy automático a cada push na branch main. Monitoramento básico com Sentry.

### Stack tecnológica:

- AWS EC2
- Nginx
- Certbot
- Docker Compose
- GitHub Actions
- Sentry

### Diferenciais a implementar:

- URL publica funcionando: <https://seudominio.com>
  - Deploy automatico a cada push via GitHub Actions
  - Monitoramento de erros em tempo real com Sentry
  - Variaveis de ambiente seguras via GitHub Secrets
- 

## Projeto 4 — Chat com documentos — RAG completo

Fase recomendada: Fase 5

Sistema de RAG completo: usuário faz upload de PDFs, o sistema faz chunking, gera embeddings e armazena num banco vetorial. O usuário pergunta algo e recebe resposta fundamentada nos documentos enviados, com streaming em tempo real. Histórico de conversas salvo no PostgreSQL. Autenticação JWT.

### Stack tecnológica:

- FastAPI
- LangChain
- Pinecone ou Chroma
- OpenAI API
- PostgreSQL
- Redis

### Diferenciais a implementar:

- Streaming de respostas do LLM em tempo real (SSE)
  - Upload multiplos formatos: PDF, DOCX, TXT
  - Citacoes das fontes na resposta: qual chunk foi usado
  - Historico de conversas persistido por usuario
- 

## Projeto 5 — Agente de IA com ferramentas reais

Fase recomendada: Fase 5 avancado

Agente autônomo que pode consultar o banco de dados (via tool), buscar informações na web, enviar e-mails, criar resumos de documentos e executar análises de dados. Back-end com FastAPI expondo o agente como API REST. Múltiplos agentes colaborando via LangGraph. Histórico de execuções persistido. Deploy na AWS com CI/CD.

### Stack tecnológica:

- LangGraph
- FastAPI
- PostgreSQL
- Tool Calling
- Docker
- AWS

**Diferenciais a implementar:**

- Agente com pelo menos 5 ferramentas reais e uteis
  - Human-in-the-loop: aprovacao humana para acoes criticas
  - Observabilidade com LangSmith: rastrear cada passo do agente
  - Multi-agente: orquestrador + especialistas por dominio
-

# Livros Essenciais por Fase

---

## Fase 1 — Python

- **Python Fluente**  
[Luciano Ramalho — Novatec/O'Reilly](#)  
O livro definitivo de Python. Escrito por um brasileiro. Referencia permanente.

- **Clean Code**  
[Robert C. Martin — Alta Books](#)  
Como escrever código legível e manutenível. Muda a forma de pensar.

## Fase 2 — Arquitetura

- **Clean Architecture**  
[Robert C. Martin — Alta Books](#)  
Como organizar sistemas que crescem sem virar bagunça. Obrigatório para pleno.

- **The Pragmatic Programmer**  
[Hunt & Thomas — Alta Books](#)  
Mentalidade de desenvolvedor profissional. Hábitos, práticas e filosofia.

## Fase 3-4 — Banco e Infraestrutura

- **Designing Data-Intensive Applications**  
[Martin Kleppmann — O'Reilly](#)  
O melhor livro sobre bancos, sistemas distribuídos e arquitetura de dados.

- **Building Microservices**  
[Sam Newman — O'Reilly](#)  
Referência em microsserviços: padrões, comunicação, deploy e trade-offs.

## Fase 5 — Inteligência Artificial

- **Hands-On Machine Learning**  
[Aurelien Geron — O'Reilly](#)  
Melhor livro prático de ML. sklearn, TensorFlow, redes neurais. Fase 5.

- **AI Engineering**  
[Chip Huyen — O'Reilly 2025](#)  
Colocar IA em produção, RAG, avaliação e MLOps. Referência atual da área.

- **Build a LLM From Scratch**  
[Sebastian Raschka — Manning 2024](#)  
Construir um GPT do zero em Python. Para entender LLMs por dentro.

## Fase 6 — Node.js

- **JavaScript: O Guia Definitivo**  
[David Flanagan — Alta Books](#)  
A bíblia do JavaScript. Leia os capítulos de ES6+ e assincronia primeiro.

● **Node.js Design Patterns**

[Mario Casciaro — Packt](#)

Padroes de projeto especificos para Node.js. Leia durante a Fase 6.

## Referencia permanente — sempre util

● **Python Testing with pytest**

[Brian Okken — O'Reilly](#)

O melhor recurso de testes em Python. Leia durante a Fase 2.

● **System Design Interview (Vol 1 e 2)**

[Alex Xu — ByteByteGo](#)

Preparacao para entrevistas de system design. Leia antes das entrevistas.



# Comunidades, Canais e Recursos Gratuitos

---

## Plataformas de prática de algoritmos

### **LeetCode** — [leetcode.com](https://leetcode.com)

Foco em Easy e Medium. 15 min por dia desde a Fase 1. Arrays, strings, recursao, BFS/DFS.

### **Beecrowd** — [beecrowd.com.br](https://beecrowd.com.br)

Plataforma brasileira. Otimo para iniciantes. Mais acessivel que LeetCode.

### **HackerRank** — [hackerrank.com](https://hackerrank.com)

Bom para praticar SQL, Python e preparar para testes de empresas.

### **Codewars** — [codewars.com](https://codewars.com)

Kata de programacao. Varios niveis de dificuldade. Bom para variar.

## Canais YouTube essenciais

### **Dunossauro (PT-BR)**

Melhor canal de Python e FastAPI em portugues. Conteudo tecnico de altissima qualidade.

### **Codigo Fonte TV (PT-BR)**

Conceitos de programacao, arquitetura e tech explicados de forma clara.

### **Rocketseat (PT-BR)**

Bom para comunidade, eventos e conteudo de back-end em geral.

### **ArjanCodes (EN)**

Python avancado, design patterns e arquitetura na pratica. Nivel muito bom.

### **Fireship (EN)**

Videos de 100 segundos sobre qualquer tecnologia. Visao geral rapida.

### **Andrej Karpathy (EN)**

Ex-OpenAI, ex-Tesla. Serie 'Neural Networks: Zero to Hero' e obrigatoria para IA.

### **ByteByteGo (EN)**

System design, arquitetura de sistemas, diagramas incriveis. Canal do Alex Xu.

### **Traversy Media (EN)**

Bom para Node.js, Express, NestJS. Tutoriais praticos e diretos.

## Plataformas de cursos e recursos gratuitos

### **deeplearning.ai**

Andrew Ng oferece dezenas de mini-cursos de IA completamente gratuitos. Os melhores do mundo.

### **roadmap.sh/backend**

Mapa visual completo do que existe no back-end. Use como referencia de onde voce esta.

### **github.com/donnemartin/system-design-primer**

O recurso mais completo de system design. Gratuito. Leia inteirinho.

### **huggingface.co/learn**

Cursos oficiais gratuitos de Hugging Face: NLP, LLMs, fine-tuning.

### **mongodb.com/learn**

Cursos oficiais gratuitos da MongoDB. Inclui certificacao.

**[skillbuilder.aws](#)**

Cursos oficiais gratuitos da AWS. Inclui preparacao para certificacoes.

**[docs.python.org/pt-br](#)**

Documentacao oficial do Python em portugues. Sempre a melhor referencia.

**[owasp.org](#)**

OWASP Top 10 e Cheat Sheets. Referencia mais completa de seguranca para devs.

## **Comunidades brasileiras de tecnologia**

**Discord Python Brasil — [pydevsbrasil.com.br](#)**

Comunidade muito ativa. Tire duvidas, veja vagas e participe de eventos.

**dev.to em portugues**

Escreva artigos sobre o que aprendeu. Isso fixa o conhecimento e gera visibilidade.

**Stack Overflow em portugues — [pt.stackoverflow.com](#)**

Para duvidas tecnicas especificas. Aprenda a fazer boas perguntas.

**GitHub — contribuicoes open source**

Comece pequeno: typo em documentacao, traducoes, issues 'good first issue'.

# Carreira e Entrevistas Técnicas

---

## Quando começar a procurar emprego

Não espere terminar o plano inteiro para aplicar. Com as Fases 0, 1 e 2 concluídas (aproximadamente 9 meses com 1h30/dia), você já pode aplicar para vagas de desenvolvedor júnior back-end Python. Com as Fases 3, 4 e 5, você está competitivo para pleno. A Fase 5 (IA) te diferencia de praticamente todos os candidatos neste nível.

## Como montar o LinkedIn

- Foto profissional: fundo neutro, rosto visível, expressão amigável
- Título: 'Desenvolvedor Back-end Python | FastAPI | em formação' (ou transição)
- Sobre: escreva o que você está construindo e para onde está indo — seja específico
- Adicione cada projeto como item de Experiência com descrição do que fez
- Habilidades: adicione Python, FastAPI, Django, PostgreSQL, Docker, Git
- Ative 'Open to Work' com visibilidade apenas para recrutadores
- Conecte-se com outros devs, recrutadores de tech e participe de grupos

## Tipos de entrevista técnica

### Coding challenges (algoritmos)

- LeetCode: foque em Easy e Medium. Pratique 15 min por dia desde a Fase 1.
- Arrays, strings, hash maps, recursão: os temas mais cobrados
- BFS/DFS em grafos e árvores: perguntas clássicas de entrevista
- Programação dinâmica básica: memoização, problema da mochila
- Comunique o raciocínio em voz alta enquanto resolve o problema
- Peça clareza antes de começar a codar. Nunca assuma nada.

### Perguntas conceituais de back-end

- Qual a diferença entre REST e GraphQL? E gRPC?
- O que é idempotência? Quais métodos HTTP são idempotentes?
- Como funciona o ACID? Quais são os níveis de isolamento?
- O que é um índice de banco? Como o B-tree funciona?
- Qual a diferença entre autenticação e autorização?
- O que é race condition? Como prevenir em Python?
- Como você escalaria uma API de 1.000 para 1.000.000 usuários?
- O que é o CAP theorem? Como ele impacta decisões de arquitetura?

### System design (cobrado em pleno)

- Como projetaria um encurtador de URLs tipo bit.ly?
- Como projetaria um feed de notícias tipo Twitter/Instagram?
- Como projetaria um sistema de notificações em tempo real?
- Estruture a resposta: requisitos -> estimativas -> componentes -> detalhar
- Pense em: banco de dados, cache, filas, CDN, load balancer
- Recurso: System Design Primer no GitHub + livros do ByteByteGo

## Entrevista comportamental (STAR)

- Metodo STAR: Situacao, Tarefa, Acao, Resultado
- Conte sobre um projeto desafiador e como resolveu o problema
- Como voce lida com prazos apertados? Priorizacao?
- Ja discordou de uma decisao tecnica da equipe? Como resolveu?
- Por que quer trabalhar nessa empresa especificamente?

## Onde procurar vagas

### LinkedIn Jobs

Filtre por 'Remoto', 'Python', 'Back-end'. Aplique em 10+ vagas por semana quando estiver pronto.

### Gupy — gupy.io

Plataforma mais usada por empresas brasileiras. Muitas vagas de tech.

### Programathor — programathor.com.br

Focada em devs. Vagas com salarios exibidos.

### Trampos — trampos.co

Focada em trabalho remoto. Boa curadoria de vagas tech.

### RemoteOK — remoteok.com

Vagas internacionais remotas. Exige ingles fluente.

### We Work Remotely — weworkremotely.com

Referencia para vagas remotas internacionais.

### GitHub Jobs / HN: Who is hiring

Mensalmente no Hacker News: 'Ask HN: Who is hiring?' — vagas de startups.

# Rotina de Estudos e Marcos de Progresso

## Rotina diária sugerida — 1h30 por dia

### 20 minutos — Revisão do dia anterior

Releia anotações, rode o código que escreveu ontem, conserte erros que ficaram. A consolidação é o que transforma estudo em memória de longo prazo. Use o método de Feynman: tente explicar o conceito como se fosse para outra pessoa.

### 50 minutos — Novo conteúdo

Vídeo-aula, leitura de documentação ou capítulo de livro. NUNCA passe um bloco inteiro só assistindo — pause a cada 10 minutos e escreva código. Sem código rodando, o conteúdo não entra.

### 20 minutos — Prática pura

Exercício no LeetCode/Beecrowd ou avance no projeto pessoal da fase. Mãos no teclado, código rodando. Comece com algo simples se travar. O importante é manter a consistência diária.

## Regras para não desistir

### Consistencia bate intensidade sempre

1h30 por dia todos os dias supera 8h num sábado e desaparecer a semana toda. O cérebro aprende em repetições espaçadas, não em maratonas.

### Não volte o vídeo antes de tentar

Quando travar num exercício, dedique 30 minutos tentando antes de buscar a solução. O erro é parte do aprendizado, não o oposto dele.

### Documente tudo no GitHub

Cada código que escrever, por menor que seja, vai pro GitHub. Você vai ver o histórico crescendo e isso é motivação concreta.

### Não pule de assunto em assunto

O maior erro de iniciantes é o 'tutorial hell': assistir um pouco de tudo sem terminar nada. Siga a sequência do plano até o fim de cada fase antes de avançar.

### Ensine o que aprendeu

Escreva artigos no dev.to ou LinkedIn sobre o que aprendeu. Isso fixa o conhecimento de forma definitiva e cria visibilidade profissional.

### Celebre os marcos

Cada projeto concluído é um marco real. Compartilhe no LinkedIn, no Discord. Comemore. A jornada é longa e o reconhecimento próprio é importante.

## Marcos de progresso — como saber onde você está

|       |                           |                                                                                            |
|-------|---------------------------|--------------------------------------------------------------------------------------------|
| Mês 1 | Primeiro commit no GitHub | Projeto de lógica + Git funcionando. Consegue criar branches, fazer commits e abrir PR.    |
| Mês 3 | Python intermediário      | Escreve scripts com POO, funções, tratamento de erros. Resolve 20+ exercícios no LeetCode. |

|        |                                            |                                                                                                           |
|--------|--------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| Mês 5  | <b>Primeira API funcionando localmente</b> | CRUD com FastAPI, banco PostgreSQL, autenticação JWT. Documentação Swagger funcionando.                   |
| Mês 9  | <b>Projeto completo com testes</b>         | API com testes cobrindo 70%+ do código. Docker Compose rodando. README bem escrito.                       |
| Mês 12 | <b>API em produção na nuvem</b>            | Projeto no ar com domínio próprio e HTTPS. CI/CD automático. Pronto para candidaturas junior.             |
| Mês 17 | <b>Portfólio principal completo</b>        | 4 projetos no GitHub. Conhecimento de banco, Docker, AWS, segurança, testes. Nível pleno em conhecimento. |
| Mês 21 | <b>IA integrada + portfólio completo</b>   | RAG funcionando em produção. Agente de IA deployado. 5 projetos. Candidaturas pleno com IA.               |

A jornada é longa, mas é muito mais linear do que parece quando você tem o mapa completo. O que diferencia quem chega do quem desiste não é talento — é a decisão diária de abrir o editor e escrever pelo menos uma linha de código. Boa sorte. Você vai chegar lá.